



brightest



Content



- Intro
- Prerequisites
- Robot Framework
- Getting Started
- Selenium

Intro





About Brightest

Mission

We aim high by creating innovative services and solutions, with a dedicated team of top-notch consultants who combine creativity and effectivity and enable our clients to realize their quality objectives.

We make high quality software evident and accessible for any organization.





About Brightest

Values

Treasure quality

Companies need to establish a quality culture and a performance-excellence mindset to meet customer expectations. With a team of passionate people and by delivering top quality services, we help companies realize these ambitions.

Excel together

We listen to the needs of our customers and handle their business as if it was our own. Software quality requires a joint and continuous effort. For that reason, we strive for long-term partnerships with our customers.



Empower people

We encourage entrepreneurship and stimulate people to act Gumption: to show guts and common sense, take initiative and go for social engagement. We strongly believe that healthy, happy and committed people will perform better.



What do we do?

Software quality is our core business

Our tailored software quality services help agile companies a digital mindset to :

- Deliver qualitative software
- Mature the quality assurance organization
- Reduce costs
- Shorten the time-to-market
- Take away all concerns regarding strategizing, plan and staffing for software quality





Who are we?

Hire for attitude and let people grow

The role of the human being in achieving quality and success has always been emphasized at Brightest. Together with the strong belief that the team is always stronger than the individual.

Part of the Gumption Group

Inspires the digital transformation of clients with a unique ecosystem and a 360° approach.



A growing scale-up founded in '16

Today we are a family of +100 dedicated consultants supported by a 15-strong office team.

Three offices in Belgium

We have three Brightest offices in Belgium. Our HQ at Kontich, one in Genk and one in Gent.



About Sepp

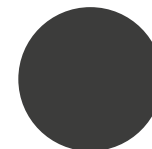
- In QA since 2013
- Since 2020 @ Brightest
- Domain Lead Functional Test Automation





Intro

- Who is who?
- Experience on automation & testing in general
- Expectations
- Why Robot Framework?



Prerequisites





Prerequisites

- Python – <https://www.python.org/>
- Robot Framework – `pip3 install robotframework`

```
seppvc@NaviBook-Pro training % pip3 install robotframework
Collecting robotframework
  Downloading robotframework-7.4.2-py3-none-any.whl (807 kB)
    

807.1/807.1 kB 7.9 MB/s eta 0:00:00


Installing collected packages: robotframework
Successfully installed robotframework-7.4.2
```

Robot Framework





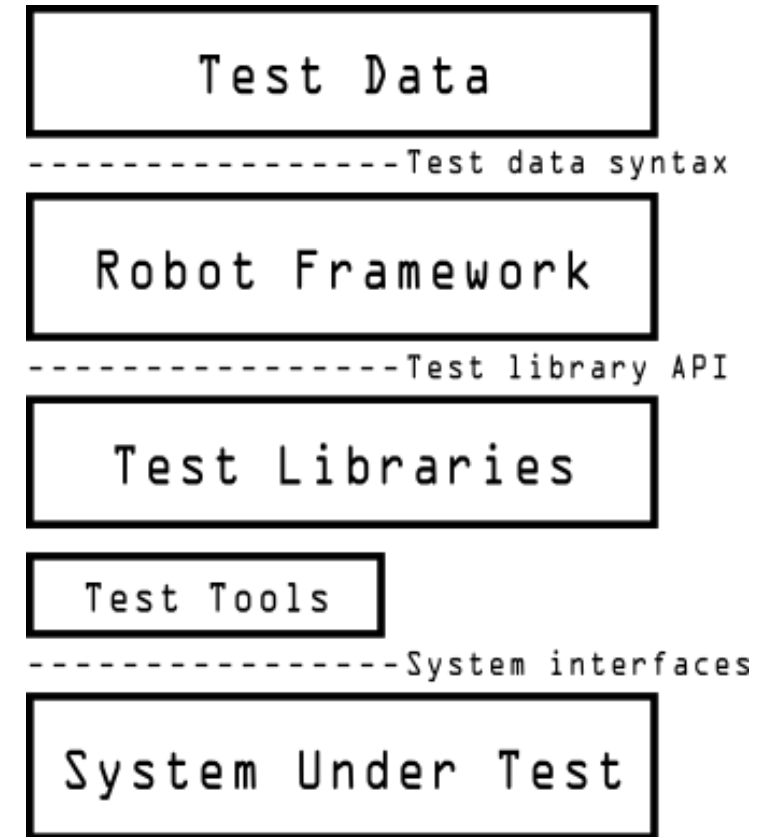
About Robot Framework

- Generic test automation framework
- Keyword/behaviour driven
- Open source
- Written in python
- .robot files



About Robot Framework

- Software abstraction layer
- Collection of libraries
- Hierarchical layering





About Robot Framework

- OS and application independent
- Implemented using Python
 - Also runs on Jython and IronPython
- Extendable via Java and Python



About Robot Framework

Keyword Driven

- Use of keywords
- Layer on top of programming syntax
- “Easier” to create and read

```
*** Test Cases ***
Login correct username and password
    Login    tomsmith    SuperSecretPassword!
    ${message}=    Get message
    Should Contain    ${message}    You logged into a secure area!
    Logout
```




About Robot Framework

Behaviour Driven

- Use of keywords
- Create test cases from behaviour perspective
- Given, When, Then, And & But prefixes are ignored by Robot Framework
- Alternative for Cucumber

```
*** Test Cases ***  
Login correct username and password  
    Given Login Page Is Open  
    When Valid Credentials Are Used  
    Then Home Page Should Be Shown
```



Keyword / Behaviour Driven

Keywords:

- Created in requirements phase
- Bound to underlying framework
 - Robot
 - Python
 - ...

Test scripts

- Created by less experienced resources

Library

- Built & maintained by test automation experts

**Getting
Started**





Syntax

Robot Framework Syntax

- Robot Framework offers unique syntax to define tests
- Keyword driven
- Double spacing (!) between keywords
- Variables not bound to one datatype

Exercise 0

1. Create a folder “Exercise0”
2. Create a new file “Exercise0.robot”
3. Add the following 3 lines of code to the file

```
*** Test Cases ***
```

```
Hello World
```

```
    Log    Hello World!
```



Running tests

- Robot files can be executed individually using command line:
`robot {directory}\{filename}.robot`
- Several robot files can be executed from a directory using:
`robot {directory}`



Reporting

- While running a robot file, progress reporting is generated in command line

```
seppvc@NaviBook-Pro training % robot Exercise0/Exercise0.robot
=====
Exercise0
=====
Hello World | PASS |
=====
Exercise0 | PASS |
1 test, 1 passed, 0 failed
=====
Output: /Users/seppvc/automation/robotframework/training/output.xml
Log:    /Users/seppvc/automation/robotframework/training/log.html
Report: /Users/seppvc/automation/robotframework/training/report.html
=====
```

- After running a robot file, 3 additional files are generated:



Reporting

report.html

Exercise0 Report

Generated
20260423 08:35:31 UTC+02:00
23 seconds ago

Summary

Status:	All tests passed
Start Time:	20260423 08:35:31.349
End Time:	20260423 08:35:31.412
Elapsed Time:	00:00:00.063
Log File:	log.html

Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	1	1	0	0	00:00:00	

Statistics by Tag	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
No Tags						

Statistics by Suite	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
Exercise0	1	1	0	0	00:00:00	

Details

All Tags Suites Search

Suite:

Test:

Include:

Exclude:

Search Clear Help



Reporting

log.html

Exercise0 Log

Generated
20260423 08:35:31 UTC+02:00
2 minutes 43 seconds ago

Statistics

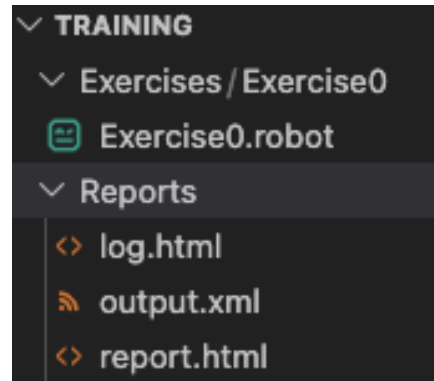
Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	1	1	0	0	00:00:00	
Statistics by Tag	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
No Tags						
Statistics by Suite	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
Exercise0	1	1	0	0	00:00:00	

Execution Log

- SUITE Exercise0	00:00:00.063
Full Name: Exercise0	
Source: /Users/seppvc/automation/robotframework/training/Exercise0/Exercise0.robot	
Start / End / Elapsed: 20260423 08:35:31.349 / 20260423 08:35:31.412 / 00:00:00.063	
Status: 1 test total, 1 passed, 0 failed, 0 skipped	
- TEST Hello World	00:00:00.001
Full Name: Exercise0.Hello World	
Start / End / Elapsed: 20260423 08:35:31.411 / 20260423 08:35:31.412 / 00:00:00.001	
Status: PASS	
- KEYWORD Builtin.Log Hello, World!	00:00:00.000
Documentation: Logs the given message with the given level.	
Start / End / Elapsed: 20260423 08:35:31.412 / 20260423 08:35:31.412 / 00:00:00.000	
08:35:31.412 INFO Hello, World!	

Running Exercise 0

1. Create a folder “Reports”



2. Navigate to this folder using your command line tool
3. Run the script from the Reports folder
ex. `robot ..\Exercise0\Exercise0.robot`



Libraries

Robot Framework libraries

- Imported automatically: BuiltIn
<http://robotframework.org/robotframework/latest/libraries/BuiltIn.html>
- Standard libraries
Collections, DateTime, OperatingSystem...
<http://robotframework.org/robotframework/>
- External libraries
AppiumLibrary, SeleniumLibrary, SikuliLibrary...
Import in the robot file under `*** Settings ***` after installing via `pip3 install`
- Custom libraries
Robot Framework file without `*** Test Cases ***` header
Can be imported via `*** Settings ***`



Libraries

Python libraries

- Python class:
 - Import keyword module
`from robot.api.deco import keyword`
 - Bind Python functions via `@Keyword`
`@keyword("Custom made keyword")`
`def custom_made_keyword(self):`
- Robot file:
 - Add python class as library
Collections, DateTime, OperatingSystem...
<http://robotframework.org/robotframework/>

Basic Methods





Course material

Examples can be found on Github:

<https://github.com/SeppVC-Brightest/Robot-Framework-Training>



Robot Framework structure

Within a robot file several sections can be defined:

*** Settings ***

This section contains references to libraries & resources available in the script
Suite Setup & Suite Teardown is also used here

*** Variables ***

Initialize any global variables to be used in the script

*** Keywords ***

Create custom keywords to be used in the script

*** Test Cases ***

Collection of the test cases to be executed when running the script

Variables

Variables are assigned to 3 datatypes:

- Scalar -> \$
- List -> @
- Dictionary -> &

```
1  *** Variables ***
2  ${GLOBALVARIABLE}  Hello World!
3
4  *** Test Cases ***
5  Assigning Variables
6      ${number}=  Set Variable  ${3}
7      ${string}=  Set Variable  String
8      ${boolean}= Set Variable  ${TRUE}
9      ${empty}=  Set Variable  ${EMPTY}
10
11  Using Lists
12      @{list}=  Create List  1  2  3
13
14  Using Dictionaries
15      &{dictionary}=  Create Dictionary  first=1  second=${2}  ${3}=third
```

NOTE: although a variable is assigned to a scalar datatype, it can still be used as a list or dictionary if it has list- or dictionary-type values



Keywords

Keywords

- Are called within test cases
- Can have multiple returns or none
- Can have multiple arguments including default arguments
- Custom keywords can be created at ***** Keywords *****

```
*** Keywords ***  
One Argument With Default Value  
    [Arguments]    ${arg}=default value  
    [Documentation] This keyword takes 0-1 arguments  
    Log    Got argument ${arg}  
  
One Required And One With Default  
    [Arguments]    ${required} ${optional}=default  
    [Documentation] This keyword takes 1-2 arguments  
    Log    Required: ${required}  
    Log    Optional: ${optional}  
  
Single Return Keyword  
    [Documentation] This keyword returns 1 value  
    RETURN    Return value  
  
Multiple Return Keyword  
    [Documentation] This keyword returns 2 values  
    RETURN    value1    ${2}
```

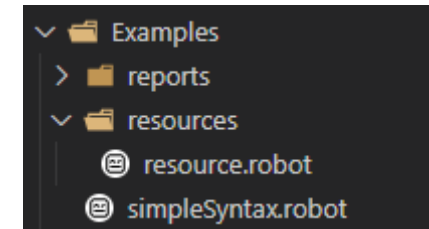


Robot Framework structure

```
1  *** Settings ***
2  Resource  ./resources/resource.robot
3
4  *** Variables ***
5  {{{
6  ${MESSAGE}  Hello World!
7
8  *** Keywords ***
9  Multiply numbers
10     [Documentation]  This keyword multiplies 2 numbers, logs and returns the result
11     [Arguments]  ${number1}  ${number2}
12     ${result}=  Evaluate  ${number1} * ${number2}
13     Log  ${result}
14     RETURN  ${result}
15
16  *** Test Cases ***
17  Test case 1
18     [Tags]  Sanity
19     ${result}=  Multiply numbers  ${4}  ${9}
20     Log  ${result}
21     # This line is a comment
22     ${result}=  Add two strings  Robot  Framework
23     Log  ${result}
```

Resource file

```
*** Keywords ***  
Add two strings  
    [Documentation] This keyword adds two strings together to form one string  
    [Arguments]     ${string1} ${string2}  
    ${result}= Catenate ${string1} ${string2}  
    RETURN ${result}
```



NOTE:

Resource files cannot contain any test cases

```
seppvc@NaviBook-Pro Reports % robot ../../Examples/resources/resource.robot  
[ ERROR ] Suite 'Resource' contains no tests or tasks.  
  
Try --help for usage information.
```



Logical operators

If Condition Is True

```
${boolean}= Set Variable ${TRUE}  
IF ${boolean}  
    Log This boolean is true  
ELSE  
    Log This boolean is false  
END
```

If Condition Is False

```
${number}= Set Variable ${7}  
IF ${number} != 7  
    Log The number was 7  
END
```

Multiple Conditions

```
${colour}= Set Variable red  
IF '${colour}'=='red' or '${colour}'=='purple'  
    Log That's a nice colour  
END
```

Setting Variables With If

```
${boolean}= Set Variable ${TRUE}  
IF ${boolean}  
    ${value}= Set Variable Variable Value 1  
ELSE  
    ${value}= Set Variable Variable Value 2  
END  
Log To Console ${value}
```



Exercise 1

Create a test script which does the following

1. Define two number variables
2. Log the largest number

Extra: define a third number variable

Extra 2: print the difference in size between the largest number and the smallest one



FOR .. IN RANGE ..

Typical for loop

Loops over several values

Step size can also be given

```
*** Test Cases ***
Only upper limit
    [Documentation]  Loops over values from 0 to 9
    FOR ${index} IN RANGE 10
        Log ${index}
    END

Start and end
    [Documentation]  Loops over values from 1 to 10
    FOR ${index} IN RANGE 1 11
        Log ${index}
    END

Also step given
    [Documentation]  Loops over values 5, 15, and 25
    FOR ${index} IN RANGE 5 26 10
        Log ${index}
    END

Arithmetic
    [Documentation]  Arithmetic with variable
    ${var}= Set Variable ${2}
    FOR ${index} IN RANGE ${var} + 1
        Log ${index}
    END
```



FOR .. IN ..

Loops over dictionaries and lists

Can also be used for list- and dictionary-like variables

Looping Over Lists

```
@{list}= Create List 1 2 3
FOR  ${entry} IN  @{list}
    Log  ${entry}
END
```

Looping Over Dictionaries

```
&{dictionary}= Create Dictionary first=1 second=${2} ${3}=third
FOR  ${key} ${value} IN  &{dictionary}
    Log  ${key}
    Log  ${value}
END
```

Breaking loop behaviour

Exit stops the code from looping

Continue stops this step of the loop and continues the next

Exiting For Loops

```
${text}= Set Variable ${EMPTY}
FOR ${var} IN one two
    IF '${var}'=='two'
        BREAK
    END
    ${text}= Set Variable ${text}${var}
END
Log ${text}
```

Continue For Loop

```
${text}= Set Variable ${EMPTY}
FOR ${var} IN one two three
    IF '${var}' == 'two'
        CONTINUE
    END
    ${text}= Set Variable ${text}${var}
END
Log ${text}
```




Nested for loops

Loops can be nested

Nested FOR Loops

```
FOR  ${i}  IN RANGE  3
    FOR  ${j}  IN RANGE  2
        Log  i=${i}, j=${j}
    END
END
```



Exercise 2

Create a test script which prints out all odd numbers from 1 to 99



Exercise 3

Print out all numbers between 1 and 100 which are:

1. Divisible by 3
2. Divisible by 5
3. Divisible by both



Passing/Failing

If a script reaches the end, Robot Framework will always return a PASS

Keywords can be added to alter the outcome

Most of them are found within the BuiltIn library and contain the word "Should"

brightest

BuiltIn

should

Keywords (50 / 107)

Should Be Empty

Should Be Equal

Should Be Equal As Integers

Should Be Equal As Numbers

Should Be Equal As Strings

Should Be True

Should Contain

Should Contain Any

Should Contain X Times

Should End With

Should Match

Should Match Regexp

Should Not Be Empty

Should Not Be Equal

Should Not Be Equal As Integers

Should Not Be Equal As Numbers

Should Not Be Equal As Strings

Should Not Be True

Should Not Contain

Should Not Contain Any

Should Not End With

Should Not Match

Should Not Match Regexp

Should Not Start With

Should Start With

Variable Should Exist

Variable Should Not Exist

Catenate

Comment

Continue For Loop

Convert To Binary

Use [Get Variable Value](#) if you need to set variables dynamically based on whether a variable exist or not.

Should Be Empty

Arguments

item

msg

=

None

Collection

str | None

Documentation

Verifies that the given item is empty.

The length of the item is got using the [Get Length](#) keyword. The default error message can be overridden with the `msg` argument.

Should Be Equal

Arguments

first

second

msg

values

ignore_case

formatter

strip_spaces

collapse_spaces

type

types

=

None

True

False

str

False

False

None

None

object

object

str | None

bool

bool

Literal['str', 'repr', 'ascii']

Literal['LEADING', 'TRAILING'] | bool

bool

Literal['AUTO'] | Any | None

Any | None

Documentation

Fails if the given objects are unequal.

See the [Controlling failure messages](#) section for information about overriding the default failure message with `msg` and `values` arguments.

The [String representations](#) section explains how `formatter` can be used for formatting values shown in failure messages.

The [String and bytes normalization](#) section explains how `ignore_case`, `strip_spaces` and `collapse_spaces` can be used for normalizing values before comparison.

The `type` and `types` arguments control optional type conversion:

- If `type` is used, the argument `second` is converted to that type. In addition to that, the argument `first` is validated to match the type.
- If `types` is used, both `first` and `second` are converted.
- Supported types are the same as supported by [automatic argument conversion](#) such as `int`, `bytes` and `list`. Also parameterized types like `list[int]` and unions like `int | float` are supported.
- When using `type`, a special value `AUTO` (case-insensitive) can be used to convert `second` to the same type that `first` has.
- Using both `type` and `types` at the same time is an error.



Passing/Failing

As soon as a “Should” condition fails, the execution of the test case stops

2 keywords are special and stop execution regardless of outcome

- Fail
- Pass Execution



Exercise 4

Make a test case which does the following

1. Get the current year
2. Fail if the year does not start with 20



Exercise 5

Make a test case which does the following

1. Get the current timestamp
2. Wait for 5 seconds
3. Get a new timestamp
4. Fail the test case if the second timestamp is before the first one
5. Print out the difference in seconds between both timestamps

Selenium





Library for browser automation

Interacts with elements on a web page

Robot Framework provides easy syntax

Must be install via pip3 install robotframework-seleniumlibrary

<http://robotframework.org/SeleniumLibrary/SeleniumLibrary.html>



Selenium uses a driver to create a browser which is browser & version specific and must be managed

Best solution is to install webdriver-manager → `pip3 install webdriver-manager`

Package this as a keyword in a resource file

```
*** Settings ***
Library    SeleniumLibrary

*** Keywords ***
Setup Chrome Driver
    [Documentation]    Automatically manages Chrome driver for current OS
    ${driver_path}=    Evaluate    __import__('webdriver_manager.chrome').chrome.ChromeDriverManager().
    install()
    RETURN    ${driver_path}

Open Browser With WebDriver Manager
    [Arguments]    ${url}    ${browser}=chrome
    ${driver_path}=    Setup Chrome Driver
    ${service}=    Evaluate    __import__('selenium.webdriver.chrome.service').webdriver.chrome.service.
    Service('${driver_path}')
    Open Browser    ${url}    ${browser}    service=${service}
```

Opening a webpage

1. Create a new robot file
2. Import our browsermanager as a Resource at `*** Settings ***`
3. Add a test case under `*** Test Case ***`
4. Add the following lines of code

Open Browser With WebDriver Manager <https://www.brightest.be> chrome

Sleep 5s

Close Browser

```
*** Settings ***  
Resource    ./selenium/helpers/browsermanager.robot  
  
*** Test Cases ***  
The First Selenium Test  
    Open Browser With WebDriver Manager    https://www.brightest.be    chrome  
    Sleep    5s  
    Close Browser
```



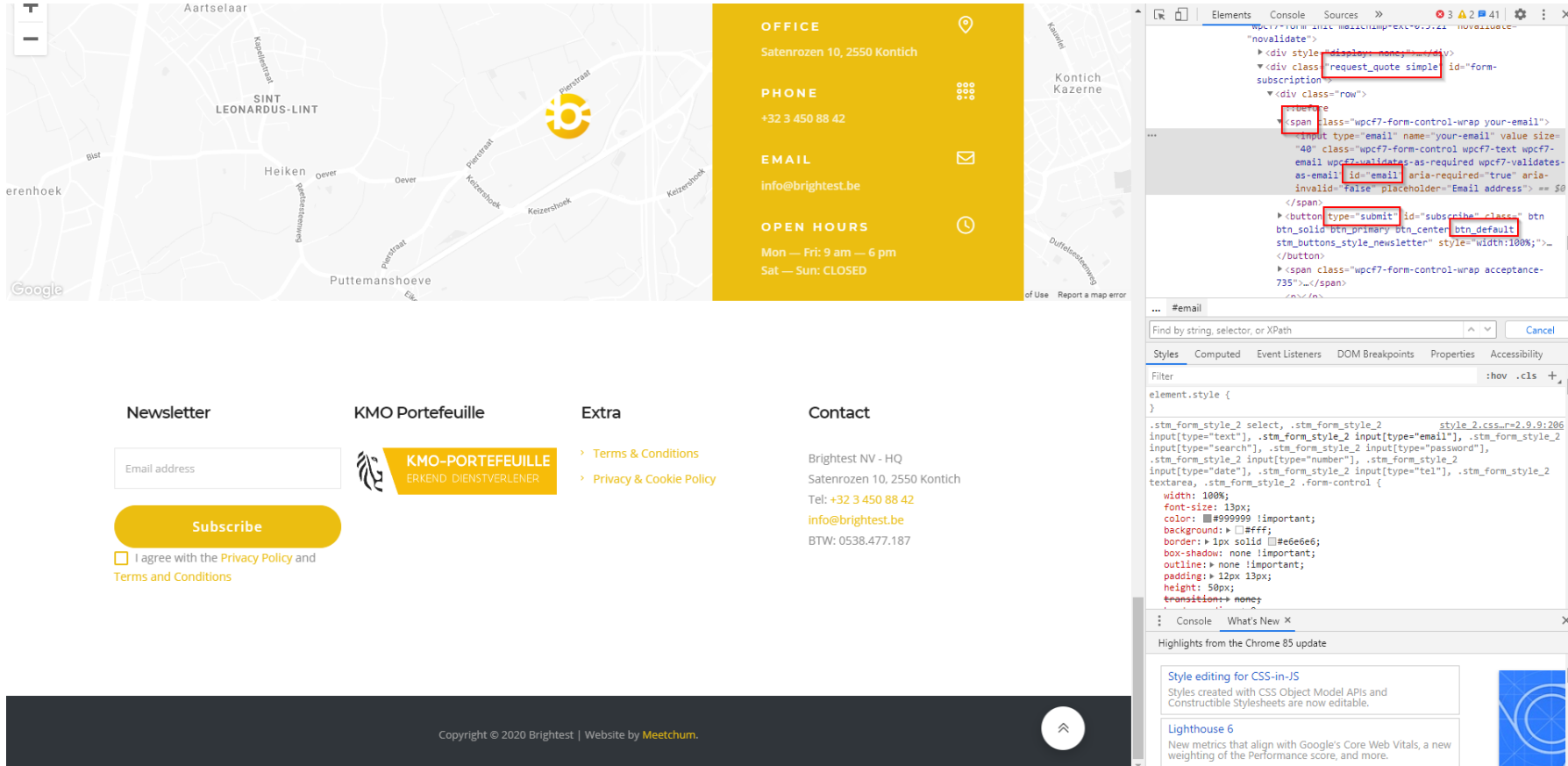
Selenium

1. Locate element
2. Interact with element

- Elements can be found on a page using several selectors
 - id
 - xpath
 - css selector
 - class
 - link
 - ...

Selectors can be built in several ways

The source code of the page is the best way to get them



The screenshot displays the brighttest website layout. At the top, a map shows the location of the office in Kontich, Belgium, with the Selenium logo highlighted. Below the map, a yellow sidebar contains contact details: Office (Satenrozen 10, 2550 Kontich), Phone (+32 3 450 88 42), Email (info@brighttest.be), and Open Hours (Mon-Fri: 9 am - 6 pm, Sat-Sun: CLOSED). The main content area features a newsletter subscription form with a text input for the email address, a 'Subscribe' button, and a checkbox for agreeing to the Privacy Policy and Terms and Conditions. To the right of the newsletter form are links for 'KMO Portefeuille', 'Extra' (Terms & Conditions, Privacy & Cookie Policy), and 'Contact' (Brightest NV - HQ, Satenrozen 10, 2550 Kontich, Tel: +32 3 450 88 42, info@brighttest.be, BTW: 0538.477.187). A red box highlights the Selenium logo on the map. Another red box highlights the 'request_quote' button in the newsletter form. A third red box highlights the 'subscribe' button in the newsletter form. On the right side of the image, a Chrome DevTools 'Elements' panel shows the DOM tree with the 'request_quote' button selected, and its HTML structure is displayed. Below the DOM tree, the 'Styles' panel shows the default styles for the button. At the bottom of the page, the brighttest logo is on the left, and the footer contains the copyright notice 'Copyright © 2020 Brightest | Website by Meetchum.' and a small circular icon on the right.

Newsletter

Email address

Subscribe

☐ I agree with the [Privacy Policy](#) and [Terms and Conditions](#)

KMO Portefeuille

KMO-PORTEFEUILLE
ERKENDE DIENSTVERLENER

Extra

- [Terms & Conditions](#)
- [Privacy & Cookie Policy](#)

Contact

Brightest NV - HQ
Satenrozen 10, 2550 Kontich
Tel: +32 3 450 88 42
info@brighttest.be
BTW: 0538.477.187

Elements

```
<div class="request_quote simple" id="form-subscription">
  <div class="row">
    <div>
      <input type="email" name="your-email" value="" size="40" class="wpcf7-form-control wpcf7-text wpcf7-email wpcf7-validates-as-required wpcf7-validates-as-email id="email" aria-required="true" aria-invalid="false" placeholder="Email address">
    </div>
    <div>
      <button type="submit" id="subscribe" class="btn btn-solid btn_primary btn_center btn_default stm_buttons_style_newsletter" style="width:100%;">
    </button>
    <div class="wpcf7-form-control-wrap acceptance-735">
    </div>
  </div>
</div>
```

Styles

```
element.style {
}
.stm_form_style_2 select, .stm_form_style_2 input[type="text"], .stm_form_style_2 input[type="email"], .stm_form_style_2 input[type="search"], .stm_form_style_2 input[type="password"], .stm_form_style_2 input[type="number"], .stm_form_style_2 input[type="date"], .stm_form_style_2 input[type="tel"], .stm_form_style_2 textarea, .stm_form_style_2 .Form-control {
  width: 100%;
  font-size: 13px;
  color: #999999 !important;
  background: #fff;
  border: 1px solid #e6e6e6;
  box-shadow: none !important;
  outline: none !important;
  padding: 12px 13px;
  height: 50px;
  transition: none;
}
```

Console

Highlights from the Chrome 85 update

Style editing for CSS-in-JS

Styles created with CSS Object Model APIs and Constructible Stylesheets are now editable.

Lighthouse 6

New metrics that align with Google's Core Web Vitals, a new weighting of the Performance score, and more.

Footer

Copyright © 2020 Brightest | Website by [Meetchum](#).



Login example

1. Go to <https://the-internet.herokuapp.com/login>
2. Enter tomsmith as username
3. Enter incorrectPassword as password
4. Click on login
5. Validate the error message to be "Your password is invalid!"
6. Close the browser



Login example

Add a second test case

1. Go to <https://the-internet.herokuapp.com/login>
2. Enter tom as username
3. Enter incorrectPassword as password
4. Click on login
5. Validate the error message to be "Your username is invalid!"
6. Close the browser



Solution

```
1  *** Settings ***
2  Resource  ./selenium/helpers/browsermanager.robot
3
4  *** Test Cases ***
5  Incorrect password
6      Open Browser With WebDriver Manager  https://the-internet.herokuapp.com/login  chrome
7      Input Text  id:username  tomsmith
8      Input Text  id:password  incorrectPassword
9      Click Element  css:Button
10     ${errorMessage}=  Get Text  css:.flash
11     Should Contain  ${errorMessage}  Your password is invalid!
12     Close Browser
13
14  Incorrect username
15     Open Browser With WebDriver Manager  https://the-internet.herokuapp.com/login  chrome
16     Input Text  id:username  tom
17     Input Text  id:password  incorrectPassword
18     Click Element  css:Button
19     ${errorMessage}=  Get Text  css:.flash
20     Should Contain  ${errorMessage}  Your username is invalid!
21     Close Browser
```




Solution

```
1  *** Settings ***
2  Resource  ./selenium/helpers/browsermanager.robot
3
4  *** Test Cases ***
5  Incorrect password
6      Open Browser With URL  http://prokuapp.com/login  chrome
7      Input Text  id:username  test
8      Input Text  id:password  123456
9      Click Element  css:button  Login
10     ${errorMessage}=  Get Text  css:div
11     Should Contain  ${errorMessage}  Invalid!
12     Close Browser
13
14  Incorrect username
15     Open Browser With URL  http://prokuapp.com/login  chrome
16     Input Text  id:username  test
17     Input Text  id:password  123456
18     Click Element  css:button  Login
19     ${errorMessage}=  Get Text  css:div
20     Should Contain  ${errorMessage}  Your username is invalid!
21     Close Browser
```



Solution - Framework

Increases reusability

Decreases maintenance

Encourages collaboration

Can be properly extended

```
▼ selenium
  ▼ helpers
    ☹ browser.robot
    ☹ browsermanager.robot
  ▼ pageObjects
    ☹ basePage.robot
    ☹ loginPage.robot
    ☹ securePage.robot
  > reports
  > tests
  ☹ userconfig.robot
  ☹ webconfig.robot
```

```
1  *** Settings ***
2  Resource  ../pageObjects/loginPage.robot
3  Resource  ../pageObjects/securePage.robot
4
5  Suite Setup  Run Keywords  Open The Internet  Go To Loginpage
6  Suite Teardown  Close browser
7
8  *** Test Cases ***
9  Correct login
10     Login With Username and Password  ${USERNAME}  ${PASSWORD}
11     ${header}=  Get Header
12     Logout
13     Should Contain  Secure Area  ${header}
14
15  Incorrect Username
16     Login With Username and Password  tom  ${PASSWORD}
17     ${error}=  Get Error Message
18     Should Contain  ${error}  Your username is invalid!
19
20  Incorrect Password
21     Login With Username and Password  ${USERNAME}  incorrectPassword
22     ${error}=  Get Error Message
23     Should Contain  ${error}  Your password is invalid!
```



Page Object Model

Models areas of a UI as objects

Usually 1 object per page

Large pages get broken down in sub pages

A Page Object is responsible for the following

find its own elements

provide functionality for those elements

Different outcomes on the same action are mapped to different methods

Page Objects to not make assertions



Exercise 6

Create a test case which does the following:

1. Navigate to <https://www.saucedemo.com/>
2. Log in as a “standard_user”
3. Add a product to the cart
4. Complete the checkout
5. Verify the “Thank you for your order!” header



brightest

Brightest NV HQ
Satenrozen 10, 2550 Kontich

Tel: +32 3 450 88 42
info@brightest.be
BTW: 0538.477.187